

Authentication & Account Management

1. Feature Name

Authentication & Account Management

2. Purpose

- **Problem Solved:** The platform needs a secure way to identify users, issue them trusted credentials for API access, and let them manage their own account data.
- **Why It Exists:** Every role-based workflow (learning paths, quizzes, classrooms, community) depends on a trusted, authenticated identity. Registration creates new student accounts, login verifies credentials, JWT access tokens authorize API calls, and refresh tokens allow sessions to continue without requiring the user to log in again.
- **Role in Application:** It is the entry point to the entire application. It produces the identity object (`User`) used by all other features, protects API endpoints via JWT, and provides self-service profile/password management.

3. What the User Can Do

Guest (unauthenticated)

- Register a new account (becomes a `Student` by default).
- Log in with email and password.

Authenticated user (Student / Instructor / Admin)

- View their own profile (name, email, avatar, bio, roles, and personal aggregate stats: paths created, modules completed, certificates).
- Update their own profile (first name, last name, avatar URL, bio).
- Change their own password (requires the current password).
- Log out (revokes all of their refresh tokens).

4. Feature Workflow

[Guest opens Login page]

→ Enters email + password

→ Frontend validates locally (`validations/authValidation.ts`)

- POST /api/v1/auth/login
- AuthController.Login → AuthService.LoginAsync
- UserManager.FindByEmailAsync + CheckPasswordAsync
- Status checks (Deleted / Inactive / Invalid)
- BuildAuthResponseAsync: generates access + refresh tokens
- Refresh token row persisted **in** RefreshTokens table
- AuthResponseDto returned inside ApiResponse envelope
- Frontend stores tokens via tokenUtils.setTokens
- authSlice stores user → AppInitializer restores session on reload

[User clicks "Log out"]

- POST /api/v1/auth/revoke (**with** Bearer access token)
- AuthService.RevokeTokenAsync marks **all** non-revoked refresh tokens **as** revoked
- Frontend clears tokens **from** localStorage **and** resets Redux state

5. How It Works Internally

Registration

Performed in `AuthService.RegisterAsync()` (backend/Services/Auth/AuthService.cs).

1. Checks `_userManager.FindByEmailAsync(dto.Email)` — if the email already exists, throws `ArgumentException("Email is already registered.")`.
2. Maps the `RegisterRequestDto` to a `User` using the `AuthMappingProfile` (sets `UserName` and `Email` from the email).
3. Calls `_userManager.CreateAsync(user, dto.Password)` — the password is hashed and stored by ASP.NET Core Identity. If identity returns errors, they are joined with " | " and thrown.
4. Calls `_userManager.AddToRoleAsync(user, "Student")` — every self-registered user is a `Student`.
5. Writes a `USER_CREATED` audit log via `IAuditLogService`.
6. Builds the auth response (access token + refresh token + user info).

Password Validation

Enforced at two levels:

- **Frontend** (`frontend/src/validations/authValidation.ts`): min 8 characters, at least one uppercase letter, at least one digit, confirm-password match.
- **Backend** (`RegisterRequestValidator, Validators/Auth/RegisterRequestValidator.cs`): required first/last name, valid email, password min 8 chars with an uppercase letter and a digit.
- **Identity policy** (`Program.cs`): `RequireDigit, RequireLowercase, RequireUppercase, RequiredLength = 8, RequireNonAlphanumeric = false, RequireUniqueEmail = true`.

Login

Performed in `AuthService.LoginAsync()`.

1. `FindByEmailAsync` — if unknown, logs `LOGIN_FAILED` audit entry and throws `UnauthorizedAccessException("Invalid credentials.")`.
2. `CheckPasswordAsync` — on failure, logs `LOGIN_FAILED` and throws the same generic message (no user enumeration).
3. Status checks — `Deleted`, `Inactive`, `Invalid` users are rejected with specific messages; each failure logs an audit entry.
4. On success, fetches the user's roles and writes a `LOGIN` audit log.
5. Calls `BuildAuthResponseAsync`.

JWT Generation

`JwtTokenGenerator.GenerateAccessToken(user, roles)` in
`backend/Authentication/Jwt/JwtTokenGenerator.cs`:

- Claims: `sub` (user id), `email`, `jti` (random GUID), `firstName`, `lastName`, plus one `ClaimTypes.Role` claim per role.
- Signed with HMAC-SHA256 using the configured `JwtSettings.Secret`.
- Contains issuer, audience, and an expiry of `JwtSettings.ExpiryMinutes` (default 4320 = 72 hours).

Refresh Token Generation

`JwtTokenGenerator.GenerateRefreshToken()` produces 64 random bytes via
`RandomNumberGenerator.Create()` (cryptographically secure), base64-encoded.

Refresh Token Storage & Rotation

In `BuildAuthResponseAsync()`:

- A new `RefreshToken` row is created with `ExpiresAt = DateTime.UtcNow.AddDays(7)` (uses `JwtSettings.RefreshTokenExpiryDays` value).
- During refresh (`RefreshTokenAsync`), the stored token is looked up by token value where `!IsRevoked && ExpiresAt > UtcNow`.
- The used refresh token is immediately set `IsRevoked = true` (rotation — each refresh invalidates the previous token) and a fresh token pair is issued.
- If the user's account is `Deleted`, `Inactive`, or `Invalid`, refresh is rejected.

Logout (Revoke)

`RevokeTokenAsync(userId)` marks all non-revoked refresh tokens for the user as `IsRevoked = true` and writes a `LOGOUT` audit log. The access token itself is not stored server-side, so it remains technically valid until expiry, but it is removed from the client.

Session Restore on the Frontend

`frontend/src/app/AppInitializer.tsx` runs on app mount:

- If there is no access token, or `tokenUtils.isTokenExpired` (JWT `exp` payload parsed and compared to `Date.now()`), tokens are cleared and auth state is reset.
- Otherwise, the JWT payload is base64-decoded and the user (`userId = sub`, `email`, `firstName`, `lastName`, `roles = role claim`) is restored into the Redux store, then notifications are fetched.

Automatic Token Refresh

`frontend/src/services/apiClient.ts` axios response interceptor:

- On any `401` response (except login/register), it calls `POST /auth/refresh` with the stored tokens, stores the new pair, retries the original request with the new access token.
- If refresh fails, tokens are cleared and the browser is redirected to `/login`.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/auth/LoginPage.tsx</code>	Login form with client-side validation and error alert
Page	<code>frontend/src/pages/auth/RegisterPage.tsx</code>	Registration form (first/last name, email, password, confirm)
Page	<code>frontend/src/pages/profile/ProfilePage.tsx</code>	Profile view/edit + change password
Service	<code>frontend/src/services/authService.ts</code>	login / register / refresh / logout API calls
State	<code>frontend/src/redux/slices/authSlice.ts</code>	loginThunk / registerThunk / logoutThunk, user + isAuthenticated
Service	<code>frontend/src/services/apiClient.ts</code>	Axios instance; Bearer header injection; 401 auto-refresh
Utils	<code>frontend/src/utils/tokenUtils.ts</code>	localStorage token persistence + JWT expiry check

Layer	File Path	Responsibility
Init	frontend/src/app/AppInitializer.tsx	Restores session from JWT on app load
Hooks	frontend/src/hooks/useAuth.ts	Convenience hook (login/register/logout + role flags)
Validation	frontend/src/validations/authValidation.ts	Client-side login/register field validation

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/AuthController.cs	POST register , POST login , POST refresh , POST revoke
Controller	backend/Controllers/UserController.cs	GET /users/me , PUT /users/me , PUT /users/me/password
Service	backend/Services/Auth/AuthService.cs	Registration, login, refresh rotation, revoke, token issuance
Service	backend/Services/User/UserService.cs	Profile get/update, password change, aggregate stats
JWT	backend/Authentication/Jwt/JwtTokenGenerator.cs	Access token + refresh token generation
JWT	backend/Authentication/Jwt/JwtSettings.cs	Secret/issuer/audience/expiry configuration
DTO	backend/DTOs/Auth/RegisterRequestDto.cs	Registration payload
DTO	backend/DTOs/Auth/LoginRequestDto.cs	Login payload
DTO	backend/DTOs/Auth/AuthResponseDto.cs	Access + refresh token + user info

Layer	File Path	Responsibility
DTO	backend/DTOs/User/UserDto.cs	UserProfileResponseDto, UpdateProfileDto, ChangePasswordDto
Validator	backend/Validators/Auth/RegisterRequestValidator.cs	Registration field validation
Validator	backend/Validators/Auth/LoginRequestValidator.cs	Login field validation
Mapping	backend/Mappings/AuthMappingProfile.cs	DTO → User mapping (UserName = Email)

8. Database Implementation

Entity/Table	Purpose	Important Relationship
AspNetUsers	Identity user (inherits IdentityUser) with FirstName, LastName, AvatarUrl, Bio, Status, IsSuperAdmin	PK id; FK target for RefreshTokens, LearningPaths, Progresses, UserClassrooms, GroupMemberships
AspNetUserRoles	Role assignment (Admin / Instructor / Student)	User ↔ IdentityRole
RefreshTokens	Server-side refresh token store	FK UserId → AspNetUsers; has Token, ExpiresAt, IsRevoked

- The `User` entity adds `UserStatus Status` (`Active`, `Inactive`, `Deleted`, `Invalid`) and `IsSuperAdmin`.
- Refresh tokens are single-use (rotated): a token is revoked as soon as it is used for a refresh.

9. Security & Authorization

- Login/register/refresh are `[AllowAnonymous]`.
- `POST /auth/revoke` and all `/users/*` endpoints require a valid JWT (`[Authorize]`).
- Passwords are hashed by ASP.NET Core Identity (`PasswordHasher`, PBKDF2/HMAC-SHA256 based) — never stored in plain text.
- Failed login attempts are logged as `LOGIN_FAILED` audit events; the user-facing error is deliberately generic (`"Invalid credentials."`) to avoid revealing whether an email exists.

- **Lockout policy:** 5 failed access attempts → 5-minute lockout (`Lockout.MaxFailedAccessAttempts = 5`, `DefaultLockoutTimeSpan = 5 min`).
- A user with `Status = Deleted/Inactive/Invalid` cannot log in or refresh tokens.
- `ChangePasswordAsync` **verifies** `CurrentPassword` **via** `UserManager.ChangePasswordAsync` **before** applying the new one.

10. Important Business Rules

1. **Unique email:** `RequireUniqueEmail` in Identity and an explicit pre-check in `RegisterAsync` block duplicate registrations.
2. **Default role:** Self-registered users are always assigned the `Student` role.
3. **Password policy:** min 8 chars, at least one uppercase letter, at least one digit (no special character required).
4. **Refresh token expiry:** 7 days; used refresh tokens are immediately revoked (rotation).
5. **Account status:** `Deleted`, `Inactive`, and `Invalid` accounts cannot log in or refresh sessions.
6. **Profile/password changes are self-scoped:** the service always uses the authenticated user's id from the JWT (`ClaimTypes.NameIdentifier`), never a client-supplied id.

11. Example of Internal Execution

Step 1: User registers

- **User Action:** New user fills the register form and submits.
- **Frontend:** `validateRegister` passes; `registerThunk` calls `authService.register` → `POST /api/v1/auth/register`.
- **Controller:** `AuthController.Register` → `AuthService.RegisterAsync(dto)`.
- **Service & DB:**
 - Email uniqueness checked.
 - User created via `UserManager.CreateAsync(user, password)` — identity hashes the password and inserts `AspNetUsers`.
 - `AddToRoleAsync(user, "Student")` inserts into `AspNetUserRoles`.
 - `BuildAuthResponseAsync` generates access + refresh tokens, inserts a `RefreshTokens` row.
- **Response:** `AuthResponseDto` inside `ApiResponse` — frontend stores tokens and logs the user in automatically.

Step 2: User logs in

- **User Action:** Returning user enters email + password.
- **Frontend:** `validateLogin` passes; `loginThunk` → `POST /api/v1/auth/login`.
- **Controller:** `AuthController.Login` → `AuthService.LoginAsync`.

- **Service & DB:** Identity verifies the password hash; status is checked; roles fetched; LOGIN audit entry written; a fresh `RefreshTokens` row is inserted.
- **Response:** access token + refresh token. The axios interceptor adds the access token as a `Bearer` header on all subsequent requests.

Step 3: Session refresh on token expiry

- The access token expires and the next API call returns `401`.
- The `apiClient` interceptor calls `POST /api/v1/auth/refresh` with the current refresh token.
- The backend revokes the used refresh token, issues a new pair, and the original request is retried with the new access token.

12. Files Involved

Frontend

- `frontend/src/pages/auth/LoginPage.tsx`
- `frontend/src/pages/auth/RegisterPage.tsx`
- `frontend/src/pages/profile/ProfilePage.tsx`
- `frontend/src/services/authService.ts`
- `frontend/src/services/apiClient.ts`
- `frontend/src/services/userService.ts`
- `frontend/src/redux/slices/authSlice.ts`
- `frontend/src/utils/tokenUtils.ts`
- `frontend/src/app/AppInitializer.tsx`
- `frontend/src/hooks/useAuth.ts`
- `frontend/src/validations/authValidation.ts`

Backend

- `backend/Controllers/AuthController.cs`
- `backend/Controllers/UserController.cs`
- `backend/Services/Auth/AuthService.cs`
- `backend/Services/User/UserService.cs`
- `backend/Authentication/Jwt/JwtTokenGenerator.cs`
- `backend/Authentication/Jwt/JwtSettings.cs`
- `backend/DTOs/Auth/RegisterRequestDto.cs`
- `backend/DTOs/Auth/LoginRequestDto.cs`
- `backend/DTOs/Auth/RefreshTokenRequestDto.cs`
- `backend/DTOs/Auth/AuthResponseDto.cs`
- `backend/DTOs/User/UserDto.cs`
- `backend/Validators/Auth/RegisterRequestValidator.cs`

- backend/Validators/Auth/LoginRequestValidator.cs
- backend/Mappings/AuthMappingProfile.cs

Database

- backend/Entities/User.cs
- backend/Entities/RefreshToken.cs
- backend/Entities/UserStatus.cs

13. Functionality

- Register a new account (auto-assigned Student role)
- Log in with email + password
- Log out (revoke all refresh tokens)
- Automatic access-token refresh via refresh-token rotation
- Session restore on page reload from stored JWT
- View own profile with aggregate stats
- Update profile (name, avatar URL, bio)
- Change password (requires current password)
- Audit logging of registration, login, failed login, logout, profile update, password change